

Implementation of combinational circuits using Verilog

//Filename: multiplexer.v

```
module mux_2to1(  
    input a, b, sel,          // Inputs  
    output y                  // Output  
);  
    assign y = (sel) ? b : a;    // If sel is 1, output b;  
    otherwise, output a  
endmodule
```

//Filename: decoder.v

```
module decoder_2to4(  
    input [1:0] in,          // 2-bit input  
    output reg [3:0] out      // 4-bit output  
);  
    always @(*) begin case (in)  
        2'b00: out = 4'b0001;  
        2'b01: out = 4'b0010;  
        2'b10: out = 4'b0100;  
        2'b11: out = 4'b1000;  
        default: out = 4'b0000; endcase  
    end  
endmodule
```

in ([1:0]): A 2-bit binary input signal, meaning it can have one of 4 values:00,01,10,or 11.

out ([3:0]): A 4-bit output signal where only one bit is set to 1 at a time, depending on the value of the input.

The always @(*) block makes it a combinational logic block that executes whenever any of the signals inside the block changes (in in this case).

A case statement is used to map input values (in) to specific output lines (out).

//Filename: combinational_circuits_tb.v

```
module testbench();  
    // Variables for 2:1 multiplexer  
    reg mux_a, mux_b, mux_sel; wire  
    mux_y;  
  
    // Variables for 2:4 decoder  
    reg [1:0] dec_in;  
    wire [3:0] dec_out;  
  
    // Instantiate 2:1 multiplexer  
    mux_2to1 MUX (  
        .a(mux_a),  
        .b(mux_b),  
        .sel(mux_sel),  
        .y(mux_y)  
    );  
  
    // Instantiate 2:4 decoder  
    decoder_2to4 DEC (  
        .in(dec_in),  
        .out(dec_out)  
    );  
  
    // Stimulate the circuits initial begin  
        // Multiplexer tests  
        $display("Testing 2:1 Multiplexer:");  
        $monitor("mux_a = %b, mux_b = %b, mux_sel = %b -> mux_y =  
%b", mux_a, mux_b, mux_sel, mux_y);  
        mux_a = 0; mux_b = 0; mux_sel = 0; #10;  
        mux_a = 0; mux_b = 1; mux_sel = 1; #10;  
        mux_a = 1; mux_b = 0; mux_sel = 0; #10;  
        mux_a = 1; mux_b = 1; mux_sel = 1; #10;  
        $display("\n");  
    end
```

```

// Decoder tests
$display("Testing 2:4 Decoder:");
$monitor("dec_in = %b -> dec_out = %b", dec_in, dec_out); dec_in = 2'b00; #10;
dec_in = 2'b01; #10; dec_in
= 2'b10; #10; dec_in =
2'b11; #10;
$display("\n");

$finish; // End simulation
end
endmodule

```

Compile and Run

To compile

```

iverilog combinational_circuits_tb.v half_adder.v full_adder.v half_sub.v
full_sub.v multiplexer.v decoder.v

```

To run vvp

a.out

```

Testing 2:1 Multiplexer:
mux_a = 0, mux_b = 0, mux_sel = 0 -> mux_y = 0
mux_a = 0, mux_b = 1, mux_sel = 1 -> mux_y = 1
mux_a = 1, mux_b = 0, mux_sel = 0 -> mux_y = 1
mux_a = 1, mux_b = 1, mux_sel = 1 -> mux_y = 1

Testing 2:4 Decoder:
dec_in = 00 -> dec_out = 0001
dec_in = 01 -> dec_out = 0010
dec_in = 10 -> dec_out = 0100
dec_in = 11 -> dec_out = 1000

combinational_circuits_tb.v:128: $finish called at 240 (1s)

```

Exercise :

1. Create a 3-to-8 Decoder
2. Create a 4 to 1 Multiplexer
3. Implement NOR gate using 2 : 1 Mux
4. Implement a 1 Bit Full Adder using Multiplexer